第四届开放原子大赛 · 2026 开源行业解决方案创新赛 · 赛道一「开源基础软件与解决方案」

给 personal agent 的 Smart-UX

agent 会写代码、会调工具、会替人办事——但它交付界面的方式，仍然是 **往聊天框里吐一段文字**。Neb 把这件事补上：agent 产出 HTML/CSS， 引擎直接渲染成**桌面原生窗**——无边框、透明、可悬浮、可穿透， 窗口即画布，界面像被画上桌面。

自研 C99 渲染引擎 · 不含浏览器内核（NO-WEB）· 零构建链 · 三平台原生二进制

定位
personal agent 的 Smart-UX 层

形态
信息窗 · 桌宠 · 数据大屏 · 流式会话 · 工具输出窗

引擎
C99 · 14 种绘制指令 · 软件/cairo/Metal 三后端

开源
MIT · github.com/asdshuaishuai/html-native

BACKGROUND

agent 什么都会做，唯独没有一张属于自己的界面。

personal agent 正在成为新的计算入口：它读文件、查资料、调工具、替人跑任务。 但交付结果的方式，今天几乎只有一种——**往对话流里打字**。数据大屏、状态面板、 桌面提醒、常驻小组件，这些"应用形态"的界面，agent 一样有能力生成， 却没有一个体面的出口。

把 HTML 搬上桌面的三条老路，各有一堵墙：

路线 A 浏览器标签

HTML 的原生归宿只有标签页。没有窗口语义——无边框、悬浮层级、点击穿透、 秒开秒毁、位置记忆，全都无从谈起。agent 交付的仍然是一张"网页"。

路线 B WebView / Electron

搬上桌面就得背浏览器内核：数千万行代码、运行时百兆起步、启动以秒计。 与 personal agent"高频生成、用完即弃"的形态天然冲突。

路线 C 原生 UI 框架

SwiftUI / Flutter 能做桌面 UI，但编译链重、产物大、改一行重新构建—— 更关键的是：agent 对它们的输出既不擅长，也无法即时预览。

HTML 恰恰是 **AI 最会写的界面语言**——它是模型训练语料中最密集、表达最流畅的 UI 语法。缺的从来不是"agent 会不会写"，而是一个**把 HTML 直接送上桌面的引擎**。

> Neb 的回答：自研 C99 渲染引擎（NO-WEB，不含浏览器内核）——agent 写 HTML/CSS， 引擎直接渲染成桌面级原生窗。不借 WebView、不需要编译、轻量秒开、增量热更新。 **对 personal agent 而言，这就是它的 Smart-UX 层**：窗口形态、呈现动效、 内容直染、声明式交互、设计令牌，一层全包。

在大赛"行业解决方案"的语境下，Neb 对应的方向是**个人智能体的界面交付层**： 无论 agent 框架如何更迭，只要它能产出文本，就能经由 Neb 获得一致的、 原生的、可热更新的桌面界面能力——这是所有 personal agent 落地到普通人桌面时 都要经过的那一层。

WHAT IT IS

一层 Smart-UX，五件事全包。

Smart-UX 不是皮肤，是 personal agent 把"会做事"变成"可感知"所需的全部界面能力。 Neb 把这层能力做成一个引擎：

壹 窗口形态

hn 窗口**默认无标题栏**——窗口即画布。几行 meta 声明出完整形态谱： 常规窗 / 消息弹卡 / 悬浮组件 / 桌宠（透明+悬浮+穿透+幽灵模式）/ 具名轻应用（销毁后回原位）。

贰 呈现动效

透明窗内容像颜料**沿羽化笔锋画上桌面**（画刷呈现）；所有窗有入场淡入 与关闭淡出；agent 推送新文档，窗口原位热更新，不闪不退。

叁 内容直染

agent 的母语直接变界面： `hn-md` （Markdown 含代码高亮/表格）、 `hn-json` 、 `hn-csv` 源文本装载即成排版好的 DOM 子树 —— 工具输出窗开箱即用。

肆 声明式交互

`hx-get/post/trigger` 点谁→拿什么→换到哪，不写 JS 也能交互； 表单编码、回车提交、 `sys://` 本地系统桥（CPU/内存/KV/剪贴板/通知） 零网络应答。

伍 设计令牌

`hn-theme` 一行注入完整设计系统（色彩/字阶/组件皮肤）， 另有 ui-kit 组件预设 include 即得——**agent 不写一行 CSS 也出精致界面**， 这是"可靠 + 符合审美"的框架层保证。

附 JS 可选

需要逻辑时内嵌 QuickJS：定时器/事件监听/canvas 2D/wasm3 纯计算模块/ fetch（带主机白名单）。不写脚本时零开销，JS 整体编译出局。

一条 agent 侧的典型交付链路：

agent 产出 HTML/CSS → daemon / CLI 推送 → **Neb 渲染成桌面原生窗** →

窗口内 hx 交互 → agent 感知结果，继续任务

这条链路里 agent 只做它最擅长的事（生成标记），界面形态、呈现、交互、主题全部由 Smart-UX 层兜底——agent 的产出第一次"开箱即像应用"，而不是一段贴在聊天记录里的代码。

ARCHITECTURE

一个引擎核心，三后端同源，零 SDK。

三层架构，边界干净：**引擎核心**（解析 → 级联 → 布局 → 14 种绘制指令的显示列表） → **rt 运行时**（QuickJS · 媒体宿主 · GPU 宿主 · 定时器 · 网络传输层） → **platform 平台胶水**（每平台一个 `hnp_*.c`，运行期 dlopen 系统 API， 编译期零 SDK 依赖）。平台只出实现，契约全在单头 `hn_platform.h`。

关键机制	说明
三后端同源	同一份显示列表，hnsoft（纯 C 软件光栅） / cairo / Metal v1（GPU 原语表） 三个执行器；GPU 覆盖不了的指令整帧静默回退，像素永远正确。macOS 真窗支持 Metal 直呈 drawable（零 CPU 位图、零回读）。
渲染质量	圆角/圆边缘 4-tap 亚像素抗锯齿；描边环按 CSS border 语义画在盒内带； 裁剪边缘按覆盖率混合——渲染口径以标准 CSS 语义为准绳。
跨架构确定性	`-ffp-contract=off` 关掉 FMA 融合；构建脚本带跨架构布局输出 逐字节比对门禁。同一文档在 macOS arm64 与 x86_64 上布局完全一致—— agent 的产出在任何机器上长得一样。
零依赖 · 零 SDK	引擎与工具纯 C99；AppKit / X11 / Metal / AVFoundation 全部运行期 dlopen； 构建与验证链只有 C99 + bash（zig cc 交叉出 5 个目标）。
无头与确定性	headless 平台使渲染/截图/事件注入全链路不触 GUI——CI 与服务器上 agent 可以完全无人化地交付与验证界面。

平台覆盖：**macOS**（arm64 真窗 + AVFoundation 媒体 + Metal 直呈）、 **Linux**（x86_64 / aarch64，X11/Xv 运行期 dlopen，桌宠形态逐字段对齐）、 **Windows**（x86_64，Win32 壳，无边框窗 + 背景拖拽）、 **headless**（无显示环境）。

引擎规模

核心约 2.6 万行 C99 · 运行时 5.3k · 平台层 5.5k

产物体积

单二进制约 1.5MB · 只链接 FreeType 与系统 libc

SCENARIOS

personal agent 的五种桌面出场方式。

全部场景由真实页面承载，仓库 `examples/` 内可直接运行 （`dist/hnapp-*.html 页面` 即开）。

场景	形态与机制
数据大屏 / 仪表盘	window 表面 + `sys://cpu|memory|disk` 周期刷新 + 进度条/条纹列表 —— agent 一份 HTML 常驻桌面做"个人驾驶舱"。
消息卡 / agent 输出	popup 秒开秒毁、同 id 热更新； `hn-md` 把工具输出 （报告/代码/表格）装载即排版，长输出自动截断保护布局。
流式会话窗	`hn-stream` 自动跟随底部、环形缓冲；agent 任务执行轨迹 实时流式上屏，用户上翻即让出、回底即恢复跟随。
桌宠 / 数字人	透明+悬浮+跨空间+穿透； `hn-mesh` 网格形变立绘 + Live2D motion 驱动（眨眼/口型/视线跟随），气泡与状态点用 keyframes 轮播。
工具输出窗 / 3D 大屏	fmt 直染窗 + canvas 2D 实时折线 + 3D 透视舞台 （preserve-3d / 透视投影 / 渐变面片）——重呈现场景同样承载。

落地形态：personal agent 框架（或任何语言写的 socket 客户端）对接常驻宿主 的 JSON-lines 协议，即可获得"open / update / close / event / shot"全套界面交付能力； 真窗模式每窗一进程、关窗即退，无安装、无常驻应用形态。 窗口位置记忆、轻应用槽位、 `.hnapp` 胶囊（文档 + 数据 + 几何 + agent 操作指令装进一个可携带 JSON 文件）让"agent 的桌面"具备持久性。

> 交付闭环
>
> open → 感知（dump/text/shot）→ 驱动（event/update）→ close

> 开发体验
>
> 保存即热更新（mtime watch ~300ms）· 无需构建链

OPEN SOURCE & ROADMAP

MIT 开源，一个人也能接手的项目。

项目以 **MIT 协议**开源（github.com/asdshuaishuai/html-native）。 第三方依赖极少且全部可审计：QuickJS（Zlib）、wasm3（MIT）、cairo/Metal/AVFoundation 为可选运行期能力；Live2D Cubism Core 为可选官方组件，按其商业授权由使用方自行提取， 不入库。文档随代码走：README（能力清单/代码地图/边界宣言）、上手指南、 平台能力矩阵、媒体契约、3D 支持度、架构决策记录与路线图，全部入库可查。

治理 可复现与可接手

构建一条命令（bash + zig cc）产出三平台产物；无 Node/npm/Xcode 工程。 代码地图按目录标注职责；关键边界（不做什么）在 README 内公开宣言， 后来者可以准确判断项目走向。

治理 标准语义为准绳

渲染行为以标准 CSS/HTML 语义为对齐目标（免单位补 px、标准选择器、 标准颜色与单位、border/裁剪语义），降低使用者与贡献者的认知成本。

近期规划（方向性）：

方向	内容
agent 生态对接	把界面交付协议封装成主流 personal agent 框架的工具插件 （function-call 一发即成窗），让"会调工具的 agent"零成本获得桌面出口。
形态与交互补全	grid 布局、select 原生控件、多屏/多表面组合、 `hn-drag` / `hn-dismiss` 弹窗交互、 `hn new` 脚手架。
持久化与分发	.hnapp 胶囊签名与资源（字体/图片）打包、 跨设备迁移、轻应用商店形态探索。
平台深化	Linux 透明窗 32 位 visual、Windows GPU 路径、 更多平台媒体宿主接入。

长期愿景：当 personal agent 成为普通人的默认计算入口， "agent → 桌面"之间需要一层轻、快、确定、开源的界面基础设施。 Neb 想成为这一层——让每一个 agent 的产出，都值得一张真正的桌面窗口。

SUBMISSION

提交物清单与可验证方式。

提交物	说明
代码仓库	github.com/asdshuaishuai/html-native（MIT； 含引擎核心/运行时/平台层/全部示例与构建脚本，README 附能力清单与边界宣言）
作品介绍文档	本文档（含项目背景 / Smart-UX 定义 / 技术架构 / 场景落地 / 治理与规划）
作品演示视频	动画演示 + 实机屏录：桌面窗秒开、画刷呈现、桌宠数字人、 数据大屏、流式会话、3D 大屏与 agent 交付闭环

评审可现场验证——所有演示均为仓库内真实运行结果：

```
$ ZIG=zig bash tools/build-multiplatform.sh     # 三平台产物，一条命令
$ dist/hnapp-macos-arm64 examples/dashboard.html        # 数据大屏（sys:// 实时刷新）
$ dist/hnapp-macos-arm64 examples/pet.html              # 桌宠数字人
$ dist/hnapp-macos-arm64 examples/fmt-demo.html         # agent 工具输出窗
$ dist/hn-daemon & dist/hn open card examples/agent-card.html --window
$ dist/hn event card --kind click --target btn          # agent 驱动闭环
```

以上每一行都可以在评审现场从干净克隆开始执行到同样的结果—— 这本身就是项目确定性与可复现性的一部分。

> 我们相信 personal agent 的终局不是聊天框里的文字，而是**长在桌面上的界面**。 Neb 把这条最后一段路修好：HTML 进，原生窗出——轻、快、确定、开源。